

Initial Design Document

Project

AlphaAgents — a multi-agent equity research system that takes a company or sector query and produces a cited, 4–6 page investment research note in under 5 minutes, with a human-in-the-loop approval step before publication.

Problem Being Solved

At AlphaDesk, a wealth-tech startup serving retail investors, senior analysts spend 4–8 hours producing a single equity research note — sourcing data from financial APIs, reading recent news, cross-referencing filings, and synthesising a structured investment thesis. This bottleneck limits coverage to a handful of companies per week and keeps retail investors under-served compared to institutional clients who receive daily research.

AlphaAgents replaces this manual pipeline with a coordinated multi-agent system. An orchestrator decomposes the query into sub-tasks; three specialised agents gather web content, financial fundamentals, and recent news in parallel; a writer agent synthesises a structured note; a critic agent reviews it for unsupported claims; and a human analyst approves or edits before publication.

What I Will Build

Component	What it does
Orchestrator	Decomposes the query into research sub-tasks; routes to the right agents in the right order
Web researcher	Browses via Tavily; fetches and summarises sources with inline citations
Financial data	Pulls fundamentals, ratios, and recent filings via yfinance; stubs Alpha Vantage; caches results
News agent	Extracts sentiment and key events from the last 7 days via NewsAPI
Writer agent	Synthesises a 4–6 page equity note: investment thesis, key risks, valuation summary, comps, recommendation
Critic agent	Reviews the draft for unsupported claims, missing risks, and factual gaps; sends back for revision if needed
HITL checkpoint	Streamlit UI for a human analyst to approve the note or request targeted revisions
Eval suite	20 sample queries with LLM-as-judge scoring on factuality, completeness, and actionability

Tech Stack

Component	Choice	Reason
Agent framework	LangGraph (Python)	Stateful graph execution, conditional routing, native tool-use support
LLM inference	Groq — Llama-3.3-70B	Free tier, fast inference, 128K context window — enough for multi-agent chaining

Web search	Tavily Search API	Free tier (1K req/month), structured results, built for LLM pipelines
Financial data	yfinance + AV stub	yfinance is fully free; Alpha Vantage free tier results cached locally to avoid 25 req/day limit
News	NewsAPI	Free tier (100 req/day); all responses cached on first fetch
Tracing	Langfuse	Free tier; full prompt + trace + eval observability; self-hostable
Vector memory	FAISS + HF embeddings	Stores past research summaries for long-term context; zero cost
UI	Streamlit	Rapid HITL interface; supports live streaming output
Deployment	HuggingFace Spaces	Free tier; Docker-based; proven deployment path

Scope

In scope: 5 agents (orchestrator + 3 data + writer + critic) · HITL Streamlit UI · 20-query eval suite with LLM-as-judge · Langfuse tracing · 3 ADRs · HuggingFace Spaces deployment.

Out of scope: Real-time tick data · paid APIs · multi-user authentication · portfolio-level analysis across multiple holdings.

Bonus if ahead of schedule: Debate agent (bull vs bear perspectives) · streaming token output in the UI · long-term vector memory of past published notes.

Week-by-Week Plan

Week	Goal
Week 1 (done by 4 Jul)	Repo setup · LangGraph state skeleton · yfinance + Tavily + NewsAPI connected · orchestrator routing to all three data agents · first raw data flowing end-to-end
Week 2 (done by 11 Jul)	Writer + Critic agents working · first end-to-end run on one company (e.g. Reliance Industries) · basic Streamlit shell visible · ADR001 committed
Week 3 (done by 18 Jul)	HITL approval UI · Langfuse tracing hooked up · 20-query eval suite running · unit + integration tests · 3 ADRs total · Milestone 1 submitted 19 Jul
Week 4 (done by 25 Jul)	HuggingFace Spaces deployment · 5-min Loom walkthrough · thinking artifact (1500-word engineering memo) · resume bullets · Milestone 2 + Final submission

Key Risks and Mitigations

Risk	Mitigation
Agent output inconsistency — agents returning malformed or uncited outputs	Pydantic structured outputs on every agent; strict system prompts; retry logic in LangGraph edges

API rate limits on all free tiers	Aggressive local caching — every API response written to JSON on first call; dev loop never re-hits the same endpoint
Scope creep toward streaming UI before core pipeline is stable	Core 5-agent pipeline end-to-end by end of Week 2; UI is a Week 3 task only
Critic-to-writer revision loop causing infinite cycles	Hard cap of 2 revision passes; if critic still fails, escalate to HITL with a flag

The goal of this project is not to replicate a Bloomberg terminal in five weeks. It is to build a well-scoped, technically honest multi-agent system that works end-to-end, is clearly documented, and demonstrates — through its ADRs, eval results, and tracing — that the builder understands how production LLM systems are designed, measured, and defended in an interview. That is the standard this design is aiming for.